

PROGRAMA JAVASCRIPT AI BACKEND DEVELOPER

Taller evaluativo — Backend profesional, seguridad y producción

Ritmo Claro API

Construye, protege, documenta y publica una API de gestión de hábitos para un cliente empresarial.

Dato	Información
Curso	JavaScript AI Backend Developer
Enfoque	Módulos 2 y 3
Grupo	JavaScript Viernes

Producto final

API backend funcional con Node.js, NestJS, PostgreSQL y Prisma, autenticada con JWT, documentada con Swagger, contenedorizada y disponible en una URL pública.

CASO EMPRESARIAL

El encargo de Ritmo Claro

Vas a trabajar como desarrollador backend para una empresa que necesita ordenar y proteger la información de sus programas de bienestar.

La organización

Ritmo Claro es una pequeña empresa que acompaña a equipos remotos en la creación de hábitos de bienestar. Sus participantes definen acciones como leer, hacer pausas activas o meditar. El equipo de soporte revisa los planes cuando una persona solicita ayuda.

El problema

Hoy los hábitos se registran en formularios y hojas compartidas. Esto produce correos duplicados, estados escritos de distintas maneras y riesgos de privacidad: una persona podría consultar o modificar información que no le pertenece. Además, el sistema solo funciona en el computador de quien lo desarrolla y no existe una documentación estable para integrarlo con una futura aplicación web.

Las personas usuarias

Actor	Necesidad	Permiso dentro del MVP
Visitante	Crear una cuenta o iniciar sesión.	Solo registro y login.
Participante USUARIO	Organizar sus propios hábitos con privacidad.	Crear, consultar, editar y eliminar sus hábitos.
Soporte ADMIN	Atender casos y revisar el estado general.	Administrar sus hábitos y consultar los hábitos de todas las personas.

La solución solicitada

Construirás el backend del producto Ritmo Claro. La API identificará a cada persona, conservará sus hábitos en PostgreSQL, aplicará permisos por rol y propiedad, rechazará datos inválidos con respuestas consistentes y publicará su contrato en Swagger. El mismo código deberá funcionar en desarrollo y producción mediante variables de entorno y Docker.

CONTRATO DEL CLIENTE

Alcance del producto mínimo viable

El MVP es deliberadamente pequeño. La evaluación se concentra en la arquitectura, la persistencia, la seguridad y la publicación del backend.

Funcionalidades obligatorias

- Registro con email único, contraseña almacenada como hash y rol USUARIO asignado por el servidor.
- Login que entrega un JWT firmado con identidad, email, rol y expiración.
- CRUD completo de hábitos, siempre limitado a la persona autenticada.
- Consulta administrativa de todos los hábitos, protegida por rol ADMIN.
- Validación de entradas, manejo uniforme de errores y mensajes comprensibles.
- Documentación Swagger en /docs con autenticación Bearer.
- Persistencia PostgreSQL mediante Prisma y migraciones versionadas.
- Repositorio GitHub sin secretos, imagen Docker y despliegue público.

Límites del MVP

- No se construye frontend, aplicación móvil ni panel administrativo visual.
- No se implementan recordatorios, rachas, marcación diaria, estadísticas ni notificaciones.
- No se incluyen pagos, funciones de inteligencia artificial ni recuperación de contraseña.
- No existe una ruta pública para elegir o cambiar el rol. El rol ADMIN se asigna fuera del flujo de registro.
- No agregues entidades o librerías que no ayuden a cumplir un requisito comprobable.

Reglas no negociables

Regla	Comportamiento comprobable
Identidad	usuarioId se obtiene del JWT; nunca se acepta desde el body.
Privacidad	Un usuario no consulta, edita ni elimina hábitos de otra persona.
Privilegio	El registro siempre crea USUARIO y la ruta global exige ADMIN.
Secretos	DATABASE_URL y JWT_SECRET quedan fuera del código y del repositorio.
Errores	La respuesta externa no expone contraseñas, secretos, stack traces ni detalles internos.

ESPECIFICACIÓN

Contrato funcional de la API

Estos datos y endpoints son el acuerdo de trabajo. Puedes mejorar nombres internos, pero no cambiar el comportamiento evaluado.

Modelo de datos mínimo

Entidad	Campos y restricciones
Usuario	id autogenerado; nombre; email único; passwordHash; rol USUARIO o ADMIN con valor inicial USUARIO; creadoEn.
Habito	id autogenerado; nombre de 3 a 120 caracteres; descripcion opcional hasta 500; estado ACTIVO, PAUSADO o ARCHIVADO; frecuencia DIARIA, SEMANAL o MENSUAL; usuarioId; creadoEn.
Relación	Un Usuario tiene muchos Habititos. Cada Habito pertenece a un solo Usuario.

Endpoints obligatorios

Método y ruta	Acceso	Resultado
POST /auth/register	Público	Crea una cuenta sin aceptar rol.
POST /auth/login	Público	Valida credenciales y entrega access_token.
POST /habitos	Con sesión	Crea un hábito del usuario autenticado.
GET /habitos	Con sesión	Lista únicamente los hábitos propios.
GET /habitos/:id	Dueño	Entrega un hábito propio.
PATCH /habitos/:id	Dueño	Actualiza solo los campos enviados.
DELETE /habitos/:id	Dueño	Elimina un hábito propio.
GET /habitos/admin/todos	ADMIN	Lista todos los hábitos sin datos sensibles.

Contrato de error

Toda respuesta de error debe exponer statusCode, timestamp, path y message. Usa 400 para datos inválidos, 401 cuando falta una identidad válida, 403 cuando existe identidad pero no permiso, 404 cuando el recurso no existe, 409 para conflictos como email repetido y 500 con mensaje genérico para fallos inesperados.

CÓMO TRABAJAR

Ruta del taller y forma de entrega

Avanza en orden. Cada parte termina con una evidencia pequeña para que puedas detectar errores antes del despliegue.

Las diez partes

Parte	Resultado principal
1	Requisitos, permisos y criterios de aceptación.
2	Proyecto NestJS y arquitectura modular.
3	Modelo Prisma, migración y persistencia PostgreSQL.
4	Registro, login, bcrypt, JWT y Passport.
5	CRUD REST de hábitos conectado a la base de datos.
6	Propiedad de recursos, roles y ruta administrativa.
7	Validaciones, errores estructurados y seguridad básica.
8	Swagger, accesibilidad documental y pruebas manuales.
9	Repositorio reproducible y contenedor Docker.
10	Base en la nube, despliegue y prueba pública.

Entregables

Producto	Debe permitir verificar
Repositorio GitHub	Código, migraciones, README, .env.example, Dockerfile, .dockerignore e historial de commits; nunca .env.
API pública	URL de producción operativa y Swagger disponible en /docs.
Pruebas	Colección de Postman o evidencia equivalente con casos exitosos y fallidos.
Documentación	Problema, alcance, instalación, variables, modelo, endpoints, decisiones y pasos de ejecución.

Reglas de trabajo

- La implementación es tuya. Puedes usar IA para comprender, revisar o diagnosticar, pero no para generar la solución completa.
- No aceptes una sugerencia que no puedas explicar o comprobar.
- Conserva evidencia por checkpoint y haz commits que representen avances identificables.
- Nunca incluyas credenciales reales en capturas, README, colección de pruebas o historial de Git.

DESARROLLO PROGRESIVO

Parte 1 Traducir el caso a requisitos

Propósito

Convertir la historia de Ritmo Claro en comportamientos observables antes de crear archivos técnicos.

Conceptos que repasas

Un requisito funcional describe lo que el sistema hace; una regla de negocio limita cómo puede hacerlo. Una historia de usuario debe terminar en un resultado que puedas probar. La matriz de permisos evita que seguridad quede como una decisión tardía.

Instrucciones paso a paso

1. Resume en README.md el problema, las personas usuarias y el valor del MVP en no más de dos párrafos.
2. Escribe al menos ocho historias de usuario repartidas entre visitante, USUARIO y ADMIN.
3. Construye una matriz que relacione cada endpoint con su actor autorizado y su resultado esperado.
4. Define casos de aceptación positivos y negativos para registro, login, CRUD, propiedad y rol.
5. Registra las decisiones que tomes sobre 403, 404 y eliminación de hábitos.
6. Apoyo con IA permitido: pide que revise ambigüedades y casos límite. Indica expresamente que no genere código ni implemente la solución.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
README.md	Presenta el problema, el alcance, los actores y la forma de comprobar el producto.
docs/decisiones.md	Conserva decisiones de alcance, permisos y criterios de aceptación con una justificación breve.

Resultado esperado

Un contrato de producto que otra persona pueda convertir en pruebas sin preguntarte qué querías decir.

Lista breve de verificación

- ☐ Cada historia tiene actor, acción y resultado verificable.
- ☐ La matriz cubre los ocho endpoints obligatorios.
- ☐ Ninguna entrada permite elegir rol o usuarioId.

Evidencia que presentas

- Enlace al commit con la primera versión del README.
- Matriz de permisos y lista de criterios de aceptación.

DESARROLLO PROGRESIVO

Parte 2 Preparar el proyecto y la arquitectura

Propósito

Crear un proyecto NestJS reproducible y una estructura donde cada pieza tenga una responsabilidad clara.

Conceptos que repasas

NestJS agrupa cada dominio en un módulo. El controller recibe HTTP y delega; el service aplica reglas; los DTO definen entradas; la inyección de dependencias conecta las piezas. Separar estas responsabilidades hace visible dónde diagnosticar un fallo.

Instrucciones paso a paso

1. Comprueba las versiones de Node y npm con `node --version` y `npm --version`.
2. Crea el proyecto con `npx @nestjs/cli@11 new ritmo-claro-api --package-manager npm` y confirma que `npm run start:dev` responde.
3. Instala el stack trabajado en clase: Config, Prisma 7 con PostgreSQL, JWT, Passport, bcrypt, class-validator, class-transformer, Swagger y Helmet.
4. Genera o crea los módulos prisma, auth y habits con sus controllers, services y carpetas dto cuando correspondan.
5. Importa los módulos funcionales desde AppModule y conserva main.ts como punto de configuración global.
6. Apoyo con IA permitido: entrega el árbol de archivos y pide una auditoría de responsabilidades. Prohíbe que escriba contenido de los archivos.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
src/main.ts	Arranca la aplicación y concentra configuración global.
src/app.module.ts	Conecta los módulos que forman la aplicación.
src/auth	Agrupar identidad, credenciales, JWT y guards de autenticación.
src/habits	Agrupar el recurso de negocio y su CRUD.
src/prisma	Exponer el acceso compartido a PostgreSQL mediante Prisma.

Resultado esperado

El servidor inicia sin errores y el árbol del proyecto permite localizar cada responsabilidad sin mezclar lógica de negocio en main.ts.

Lista breve de verificación

- ☐ El proyecto arranca en modo desarrollo.
- ☐ AppModule importa los módulos necesarios.
- ☐ Puedes explicar controller, service, module, DTO e inyección.

Evidencia que presentas

- Captura o registro de la respuesta inicial local.
- Árbol de src y commit del proyecto base.

DESARROLLO PROGRESIVO

Parte 3 Modelar y persistir los datos

Propósito

Representar usuarios y hábitos en PostgreSQL y comprobar que las restricciones sobreviven al reinicio de la aplicación.

Conceptos que repasas

El schema de Prisma describe modelos, campos, enums y relaciones. Una migración versiona cambios reales en la base; prisma generate crea el cliente que usa la aplicación. Son operaciones distintas y ambas deben ser reproducibles.

Instrucciones paso a paso

1. Inicializa Prisma para PostgreSQL y configura el cliente generado en `src/generated/prisma` según la versión 7 trabajada en clase.
2. Modela Usuario y Habito con sus restricciones, valores iniciales y relación uno a muchos.
3. Configura `DATABASE_URL` fuera del código y documenta su nombre en `.env.example` sin credenciales.
4. Ejecuta `npx prisma format`, `npx prisma migrate dev --name modelo_inicial` y `npx prisma generate`.
5. Usa `npx prisma studio` para verificar tablas, relación, email único y enums.
6. Detén y reinicia la aplicación para confirmar que los datos permanecen.
7. Apoyo con IA permitido: pide una auditoría de cardinalidades, defaults y restricciones; solicita observaciones, no un schema reescrito.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
<code>prisma/schema.prisma</code>	Define el modelo relacional y sus restricciones.
<code>prisma.config.ts</code>	Carga la configuración de Prisma y la conexión según Prisma 7.
<code>prisma/migrations</code>	Versiona la estructura que también deberá aplicar producción.
<code>src/prisma/prisma.service.ts</code>	Construye y comparte el cliente de acceso a datos.
<code>src/generated/prisma</code>	Contiene el cliente generado; no se edita a mano.

Resultado esperado

La base contiene las dos tablas relacionadas, rechaza correos duplicados y conserva los datos después de reiniciar el backend.

Lista breve de verificación

- ☐ Puedes señalar la clave primaria y la clave foránea.
- ☐ La migración está versionada en Git.
- ☐ Generar el cliente no se confunde con aplicar una migración.

Evidencia que presentas

- Diagrama simple de la relación y captura de Prisma Studio sin datos sensibles.
- Commit que incluye schema y migración inicial.

DESARROLLO PROGRESIVO

Parte 4 Implementar registro y login

Propósito

Permitir que una persona cree una cuenta y demuestre su identidad mediante un token firmado.

Conceptos que repasas

bcrypt genera un hash no reversible para verificar contraseñas sin almacenarlas en texto plano. JWT contiene afirmaciones firmadas y una expiración; Passport extrae y valida el token antes de que la petición llegue al caso de uso.

Instrucciones paso a paso

1. Define DTO separados para registro y login. El registro admite nombre, email y password; nunca rol.
2. En el servicio de autenticación, rechaza email repetido con 409 y guarda solo el hash de la contraseña.
3. Implementa un mensaje genérico para credenciales inválidas que no revele si el email existe.
4. Firma un JWT con sub, email, rol y expiración de una hora usando JWT_SECRET desde el entorno.
5. Configura la estrategia Passport, el guard de JWT y el decorador que entrega el usuario autenticado.
6. Prueba registro válido, duplicado, login válido, contraseña incorrecta, token alterado y ausencia de token.
7. Apoyo con IA permitido: describe tu flujo y pide una lista de riesgos o verificaciones faltantes. No solicites implementación.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
register.dto.ts y login.dto.ts	Definen los datos permitidos para cada operación.
auth.service.ts	Aplica registro, hash, verificación de credenciales y firma del JWT.
auth.controller.ts	Expone las dos rutas públicas y delega al servicio.
jwt.strategy.ts	Valida el token y transforma su payload en identidad autenticada.
jwt-auth.guard.ts	Impide el acceso cuando falta un token válido.
usuario-actual.decorator.ts	Entrega la identidad validada sin confiar en el body.

Resultado esperado

El registro nunca devuelve passwordHash; el login válido entrega access_token y las credenciales inválidas producen 401 sin filtrar información.

Lista breve de verificación

- ☐ Un usuario nuevo nace con rol USUARIO.
- ☐ JWT_SECRET no aparece en el código ni en Git.
- ☐ El token vencido o alterado es rechazado.

Evidencia que presentas

- Respuestas de Postman para registro, duplicado, login y credenciales inválidas.
- Explicación del contenido y la expiración del JWT sin mostrar secretos.

DESARROLLO PROGRESIVO

Parte 5 Construir el CRUD de hábitos

Propósito

Completar el flujo REST principal con persistencia real y una separación clara entre HTTP, reglas y datos.

Conceptos que repasas

En una API REST, método y ruta expresan la operación sobre un recurso. El controller interpreta la petición; el service decide qué se puede hacer y consulta Prisma. PATCH modifica solo campos presentes y los códigos HTTP deben coincidir con el resultado.

Instrucciones paso a paso

1. Crea los DTO de creación y actualización sin aceptar id, usuarioId ni campos automáticos.
2. Implementa POST /habitos y asigna el dueño a partir de la identidad autenticada.
3. Implementa GET /habitos y filtra por el usuario del JWT.
4. Implementa consulta, actualización parcial y eliminación por id, manteniendo las reglas de existencia.
5. Conserva las consultas Prisma dentro del service y deja el controller enfocado en HTTP.
6. Comprueba los valores iniciales de estado y frecuencia definidos por el modelo.
7. Apoyo con IA permitido: comparte el resultado de una prueba fallida y pide hipótesis ordenadas por capa; no pidas que escriba el método.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
crear-habito.dto.ts	Declara los campos permitidos y sus reglas de entrada.
actualizar-habito.dto.ts	Permite cambios parciales sin abrir campos protegidos.
habitos.controller.ts	Define rutas, parámetros, identidad y respuestas HTTP.
habitos.service.ts	Aplica reglas, consultas, filtros por dueño y operaciones Prisma.
habitos.module.ts	Registra controller, service y dependencias del dominio.

Resultado esperado

Una persona autenticada completa crear, listar, consultar, editar y eliminar sus hábitos, y los cambios permanecen en PostgreSQL.

Lista breve de verificación

- ☐ Los cinco endpoints del CRUD funcionan.
- ☐ El controller no consulta Prisma directamente.
- ☐ PATCH conserva los campos no enviados.

Evidencia que presentas

- Secuencia de Postman con creación, listado, actualización y eliminación.
- Captura de persistencia y commit del CRUD completo.

DESARROLLO PROGRESIVO

Parte 6 Aplicar roles y propiedad

Propósito

Impedir accesos entre usuarios y reservar la consulta global para el personal autorizado.

Conceptos que repasas

Autenticación responde quién eres; autorización decide qué puedes hacer. RBAC compara el rol del token con el requerido por la ruta. Ownership compara la identidad autenticada con el dueño del recurso. Son controles distintos y ambos deben probarse.

Instrucciones paso a paso

1. Protege todas las rutas de hábitos con JwtAuthGuard.
2. Verifica propiedad antes de entregar, editar o eliminar un hábito por id.
3. Define el decorador de roles y un RolesGuard que lea la metadata de la ruta.
4. Agrega GET /habitos/admin/todos con guard de JWT y requisito ADMIN. Ubica la ruta para que no sea confundida con /habitos/:id.
5. Limita la respuesta administrativa a datos necesarios y excluye siempre passwordHash.
6. Prueba con usuario A, usuario B y ADMIN. A crea; B intenta consultar, editar y eliminar; ADMIN consulta el listado global.
7. Apoyo con IA permitido: entrega tu matriz de permisos y resultados; pide que identifique huecos de autorización sin generar guards.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
roles.decorator.ts	Declara los roles permitidos como metadata de una ruta.
roles.guard.ts	Compara metadata con el rol de la identidad autenticada.
jwt-auth.guard.ts	Exige una identidad válida antes de evaluar permisos.
habitos.service.ts	Comprueba propiedad y limita consultas por usuario.
habitos.controller.ts	Combina guards y decoradores en cada ruta protegida.

Resultado esperado

Los intentos entre usuarios reciben 403, la falta de token recibe 401 y solo ADMIN obtiene el listado global sin datos sensibles.

Lista breve de verificación

- ☐ usuarioId nunca llega desde el body.
- ☐ La ruta administrativa exige autenticación y rol.
- ☐ Cambiar un rol requiere iniciar sesión de nuevo para emitir otro token.

Evidencia que presentas

- Pruebas comparables de usuario A, usuario B y ADMIN.
- Matriz final de permisos con códigos observados.

DESARROLLO PROGRESIVO

Parte 7 Validar entradas y unificar errores

Propósito

Rechazar datos inválidos antes de ejecutar reglas y responder fallos sin exponer detalles internos.

Conceptos que repasas

Los DTO con class-validator convierten requisitos en validaciones ejecutables. ValidationPipe aplica esas reglas y transforma entradas. Un filtro de excepciones mantiene un contrato estable; un error inesperado se registra en el servidor y responde con un mensaje externo genérico.

Instrucciones paso a paso

1. Valida registro: nombre de mínimo 2 caracteres, email válido y password de mínimo 8.
2. Valida hábitos: nombre entre 3 y 120, descripción opcional hasta 500 y enums válidos para frecuencia y estado.
3. Configura ValidationPipe global con whitelist y transform. Decide y documenta si rechazas campos adicionales.
4. Implementa el formato statusCode, timestamp, path y message para errores conocidos.
5. Traduce los casos de negocio a 400, 401, 403, 404 y 409 según el contrato.
6. Para fallos inesperados, responde 500 genérico y conserva el detalle solo en logs.
7. Activa Helmet y confirma que ningún error devuelve stack, passwordHash, JWT_SECRET o DATABASE_URL.
8. Apoyo con IA permitido: entrega respuestas de error anonimizadas y pide detectar inconsistencias o fugas; no solicites el filtro completo.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
dto/*.dto.ts	Ejecutan reglas de forma, longitud, obligatoriedad y enum.
http-exception.filter.ts	Normaliza el contrato y oculta detalles internos.
src/main.ts	Registra pipe, filtro, Helmet y configuración global.
habitos.service.ts y auth.service.ts	Lanzan errores de negocio con el código adecuado.

Resultado esperado

Entradas inválidas no llegan a la lógica; todos los errores conservan la misma forma y un fallo interno no revela información sensible.

Lista breve de verificación

- ☐ Nombre corto, email inválido y enum inventado producen 400.
- ☐ Email repetido produce 409.
- ☐ La respuesta 500 es genérica y el log conserva el diagnóstico.

Evidencia que presentas

- Muestra de errores 400, 401, 403, 404, 409 y 500 controlado.
- Explicación de la diferencia entre validación, regla de negocio y fallo inesperado.

DESARROLLO PROGRESIVO

Parte 8 Documentar y probar la API

Propósito

Convertir el comportamiento de la API en un contrato navegable y demostrar tanto éxitos como rechazos.

Conceptos que repasas

OpenAPI describe rutas, seguridad, entradas y respuestas; Swagger presenta ese contrato de forma interactiva. Una prueba manual útil incluye camino feliz, límites, permisos y errores. La accesibilidad documental exige lenguaje claro, orden lógico y evidencia comprensible sin depender solo del color.

Instrucciones paso a paso

1. Configura Swagger en /docs con título, versión, agrupación por auth y hábitos, y autenticación Bearer.
2. Documenta qué recibe y devuelve cada endpoint, junto con los códigos relevantes.
3. Revisa que los nombres de campos, enums y rutas coincidan con la implementación.
4. Crea una colección de Postman con variables baseUrl y token; no guardes secretos ni tokens vigentes al entregarla.
5. Ejecuta la matriz de pruebas de la página 16 con dos usuarios y un ADMIN.
6. Verifica accesibilidad básica: títulos descriptivos, orden de lectura coherente, mensajes claros y evidencia que incluya texto o código de estado, no solo colores.
7. Apoyo con IA permitido: comparte la lista de pruebas y pide detectar requisitos sin cobertura. Prohíbe generar la implementación.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
src/main.ts	Publica el documento OpenAPI y habilita Bearer.
auth.controller.ts y habits.controller.ts	Agrupar y describen las operaciones y respuestas.
dto/*.dto.ts	Mantienen alineadas validación y documentación de entradas.
postman/Ritmo-Claro.postman_collection.json	Conserva las pruebas reproducibles sin credenciales.
README.md	Explica cómo ejecutar la colección y abrir Swagger.

Resultado esperado

Swagger permite recorrer el contrato completo y la colección demuestra los flujos positivos, negativos, de propiedad y de rol.

Lista breve de verificación

- ☐ Los ocho endpoints aparecen en /docs.
- ☐ Authorize permite probar rutas protegidas.
- ☐ La evidencia identifica el resultado por texto y status HTTP.

Evidencia que presentas

- Captura completa de /docs y colección exportada.
- Resultados de la matriz de pruebas con fecha y entorno.

DESARROLLO PROGRESIVO

Parte 9 Preparar GitHub y Docker

Propósito

Dejar un proyecto reproducible, sin secretos y listo para ejecutarse igual en otra máquina.

Conceptos que repasas

Una variable de entorno separa configuración y código. Una imagen Docker es la plantilla inmutable; un contenedor es una instancia en ejecución. El repositorio debe incluir lo necesario para reconstruir la app, pero excluir dependencias generadas, compilados y secretos.

Instrucciones paso a paso

1. Completa `.env.example` con `DATABASE_URL`, `JWT_SECRET` y `PORT` usando valores ficticios o instrucciones, nunca credenciales.
2. Confirma que `.gitignore` excluye `.env`, `node_modules` y `dist`, pero conserva `prisma/migrations`.
3. Ejecuta `npm run build` y corrige errores antes de crear la imagen.
4. Crea un `Dockerfile` que instale de forma reproducible, genere Prisma, compile, aplique migraciones de producción al arrancar y ejecute el build.
5. Crea `.dockerignore` para excluir `.env`, `.git`, `node_modules`, `dist` y `logs`.
6. Si tienes Docker local, prueba `docker build -t ritmo-claro-api .` y ejecuta la imagen con `--env-file .env`.
7. Revisa `git status`, el historial y el repositorio remoto para confirmar que no existe `.env`.
8. Apoyo con IA permitido: comparte el árbol y las instrucciones Docker; pide detectar archivos ausentes, secretos o pasos no reproducibles, sin solicitar un `Dockerfile` completo.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
<code>.env</code> y <code>.env.example</code>	Separan valores privados del contrato público de configuración.
<code>.gitignore</code> y <code>.dockerignore</code>	Evitan publicar o empaquetar archivos innecesarios y secretos.
<code>Dockerfile</code>	Describe instalación, generación, build, migración y arranque.
<code>package.json</code> y <code>package-lock.json</code>	Fijan scripts y dependencias reproducibles.
<code>README.md</code>	Documenta instalación local, variables, build y ejecución.

Resultado esperado

Otra persona puede clonar, configurar y ejecutar el proyecto; el build pasa y la imagen no contiene el archivo `.env`.

Lista breve de verificación

- ☐ `npm run build` termina sin errores.
- ☐ El repositorio incluye migraciones y excluye secretos.
- ☐ La imagen arranca con configuración externa.

Evidencia que presentas

- URL del repositorio e historial de commits.
- Salida del build y, si aplica, prueba local del contenedor.

DESARROLLO PROGRESIVO

Parte 10 Desplegar y comprobar producción

Propósito

Publicar la API con base persistente, configuración privada y un flujo verificable desde una URL externa.

Conceptos que repasas

El deploy reúne repositorio, imagen, variables de entorno, migraciones y base de datos. El mismo código debe funcionar en local y nube; solo cambia la configuración. Los logs ayudan a ubicar si el fallo ocurrió al construir, migrar, conectar o arrancar.

Instrucciones paso a paso

1. Crea un proyecto PostgreSQL en Supabase y conserva la cadena del Session Pooler como DATABASE_URL privada.
2. Publica el repositorio y crea en Render un Web Service con runtime Docker.
3. Configura DATABASE_URL y un JWT_SECRET largo en el entorno de Render. Permite que la plataforma suministre PORT.
4. Observa los logs hasta confirmar instalación, generación, build, migraciones y arranque.
5. Abre la URL pública en /docs y repite registro, login, creación y listado de un hábito.
6. Comprueba 401 sin token, 403 con usuario normal en la ruta administrativa y acceso exitoso con ADMIN.
7. Reinicia o vuelve a desplegar y confirma que los datos persisten.
8. Actualiza README con URL, pasos de prueba, variables requeridas y una nota sobre el posible tiempo de activación del plan gratuito.
9. Apoyo con IA permitido: pega un extracto sin secretos del primer error de los logs y pide hipótesis de diagnóstico; no pidas reescribir la aplicación.

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
Dockerfile	Define la construcción y el arranque que ejecuta la plataforma.
prisma/migrations	Aplica en producción la estructura versionada de la base.
src/main.ts	Lee PORT del entorno y arranca el servidor en el contenedor.
.env.example	Enumera las variables sin valores privados.
README.md	Entrega URL, instrucciones de prueba y decisiones de despliegue.

Resultado esperado

La API responde desde una URL HTTPS pública, conserva datos en PostgreSQL y permite demostrar autenticación, permisos, validación y documentación.

Lista breve de verificación

- ☐ Swagger público carga y coincide con la API.
- ☐ El flujo completo funciona contra producción.
- ☐ La URL y el repositorio no exponen secretos.

Evidencia que presentas

- URL pública, URL de /docs y enlace al repositorio.
- Capturas o registro del smoke test y de los permisos 401 y 403.

COMPROBACIÓN INTEGRAL

Matriz de pruebas manuales obligatorias

Ejecuta los casos en local y repite los marcados como producción. Registra fecha, entorno, status observado y una evidencia legible.

Caso	Acción	Resultado esperado
Registro válido	Crear cuenta con datos correctos.	201 y usuario sin passwordHash.
Email repetido	Registrar dos veces el mismo email.	409 con contrato de error.
Login válido	Usar credenciales correctas.	Token firmado y con expiración.
Login inválido	Usar email o password incorrectos.	401 con mensaje genérico.
Entrada inválida	Enviar nombre corto o enum inexistente.	400 con mensaje comprensible.
Sin autenticación	Consultar /habitos sin token.	401.
Crear y listar	Crear un hábito y consultar la lista propia.	201 y luego 200 con el registro.
Actualización parcial	Cambiar un solo campo.	Los demás campos se conservan.
Recurso inexistente	Consultar un id que no existe.	404.
Propiedad	Usuario B intenta ver, editar y borrar el hábito de A.	403 en cada intento.
RBAC	USUARIO llama /habitos/admin/todos.	403.
Ruta administrativa	ADMIN llama /habitos/admin/todos.	200 sin datos sensibles.
Eliminación	El dueño elimina y vuelve a consultar.	Éxito al borrar y 404 después.
Producción	Repetir registro, login, creación y permisos en la URL pública.	Mismo contrato que en local.

Registro sugerido de evidencia

Usa una tabla o colección donde cada prueba conserve nombre, entorno, fecha, entrada resumida, status observado, resultado y enlace a la captura. Oculta tokens, contraseñas, cadenas de conexión y cualquier valor real de entorno.

Criterio de cierre

No marques una parte como terminada si solo probaste el camino feliz. Los rechazos 400, 401, 403, 404 y 409 también demuestran que el producto cumple.

100 PUNTOS

Rúbrica de evaluación

La calificación se basa en comportamiento comprobable, calidad técnica y capacidad de explicar las decisiones. Esta primera página reúne 70 puntos.

Criterio	Pts	Desempeño completo	Descuentos frecuentes
Dominio y modelo de datos	15	Historia, alcance y contratos coherentes; relación Usuario-Habito; enums, unicidad y migración versionada.	Campos sin propósito; relación incorrecta; datos en memoria; schema sin migración.
Arquitectura y CRUD REST	15	Módulos claros; controllers delgados; services con reglas; cinco operaciones CRUD persistentes y status coherentes.	Prisma en controllers; rutas incompletas; PATCH destructivo; responsabilidades mezcladas.
Autenticación	15	Registro sin rol; hash bcrypt; login seguro; JWT con expiración; Passport y guard funcionales; datos sensibles ausentes.	Password en texto o respuesta; secreto en código; mensajes que revelan usuarios; token sin validar.
Autorización y propiedad	15	Rutas protegidas; dueño obtenido del JWT; aislamiento entre usuarios; RBAC y listado global solo ADMIN.	usuarioId desde body; fugas entre usuarios; ruta administrativa abierta; roles confiados al cliente.
Validación errores y seguridad	10	DTO ejecutables; ValidationPipe; contrato uniforme; 500 genérico; Helmet; códigos 400, 401, 403, 404 y 409 correctos.	Types sin validación; errores crudos; códigos incoherentes; datos internos expuestos.

30 PUNTOS RESTANTES

Rúbrica de evaluación

Los nueve criterios suman exactamente 100 puntos.

Criterio	Pts	Desempeño completo	Descuentos frecuentes
Swagger pruebas y accesibilidad	10	Ocho endpoints documentados; Bearer; contratos claros; colección con casos positivos, negativos, dos usuarios y ADMIN; evidencia comprensible sin depender del color.	Docs desactualizadas; solo camino feliz; sin prueba de ownership; capturas ambiguas.
Git README e IA	10	Commits identificables; README reproducible; .env.example; decisiones registradas; uso de IA limitado a comprensión, revisión o diagnóstico.	Un solo commit; instalación incompleta; código no explicado; prompts que generan la solución.
Docker y despliegue	10	Build limpio; imagen reproducible; migraciones; Supabase; Render; variables privadas; URL pública persistente.	Solo localhost; puerto fijo; base efímera; secretos publicados; Dockerfile incompleto.

Cómo se asignan los puntos

- Completo: cumple el comportamiento y lo demuestra con evidencia reproducible.
- Parcial: funciona de forma incompleta, depende de datos manuales o falta evidencia importante.
- No verificable: el código existe, pero no compila, no se puede ejecutar o no se puede explicar.

Interpretación del resultado

Rango	Interpretación
90 a 100	Producto completo, seguro, publicado y defendido con criterio técnico.
80 a 89	Cumple el flujo principal con fallos menores o evidencia incompleta.
70 a 79	Funciona parcialmente, pero tiene vacíos relevantes en seguridad, pruebas o producción.
60 a 69	Demuestra conceptos aislados; el flujo completo todavía no es confiable.
Menos de 60	No permite verificar las competencias centrales de los módulos 2 y 3.

ANTES DE ENVIAR

Lista final de entrega

Haz una última revisión completa. La entrega debe permitir que otra persona ejecute, pruebe y comprenda el proyecto sin depender de explicaciones privadas.

Producto y funcionamiento

- ☐ La URL pública responde y /docs carga correctamente.
- ☐ Registro, login y CRUD funcionan en producción.
- ☐ JWT protege las rutas y el aislamiento entre usuarios está demostrado.
- ☐ La ruta administrativa rechaza USUARIO y acepta ADMIN.
- ☐ Los datos persisten después de reiniciar o volver a desplegar.

Repositorio y documentación

- ☐ El repositorio abre, compila y contiene package-lock.json.
- ☐ prisma/migrations está versionada; node_modules, dist y .env no lo están.
- ☐ .env.example enumera DATABASE_URL, JWT_SECRET y PORT sin valores reales.
- ☐ README explica problema, alcance, arquitectura, instalación, variables, migración, ejecución, pruebas, Docker, deploy y URLs.
- ☐ La colección de Postman no guarda tokens ni credenciales vigentes.
- ☐ Las capturas no exponen secretos ni información personal innecesaria.

SUSTENTACIÓN

Preguntas para la defensa técnica

Responde con ejemplos de tu propio proyecto. La instructora puede pedir una prueba o un cambio pequeño para comprobar comprensión y autoría.

N.º	Pregunta
1	Sigue una petición a POST /habitos desde que llega hasta que PostgreSQL confirma el registro.
2	¿Por qué el controller no debería contener consultas directas a Prisma?
3	¿Qué diferencia existe entre prisma migrate y prisma generate?
4	¿Qué impide que dos cuentas usen el mismo email? ¿En qué capa se comprueba?
5	¿Por qué bcrypt usa hash y no cifrado reversible para la contraseña?
6	¿Qué datos contiene tu JWT, cuánto dura y qué no debería contener?
7	Explica con una prueba la diferencia entre 401 y 403.
8	¿Por qué usuarioId se obtiene del token y no del body?
9	¿Qué ocurre si cambias el rol en la base, pero sigues usando un token emitido antes?
10	¿Qué hacen whitelist y transform en tu ValidationPipe?
11	¿Cómo evitas que un error inesperado exponga información interna?
12	¿Cómo demuestras que Swagger coincide con la implementación real?
13	¿Qué diferencia hay entre .env y .env.example?
14	¿Qué diferencia hay entre una imagen Docker y un contenedor?
15	¿Por qué producción necesita las migraciones del repositorio?
16	Si Render inicia pero no conecta a PostgreSQL, ¿qué revisarías primero y por qué?
17	Si usuario B logra ver un hábito de A, ¿en qué capas investigarías el fallo?
18	Muestra una sugerencia de IA que rechazaste o corregiste y explica cómo la verificaste.

Cierre

El objetivo es demostrar que puedes construir un backend completo y explicar por qué funciona. Si una parte falla durante la presentación, identifica la capa, formula una hipótesis y usa la evidencia para diagnosticarla.